mt5-mac 0.3.0

MetaTrader 5 native Python library for macOS - control MT5 from Python scripts

```
pip install mt5-mac
```

pypi v0.3.0 | license MIT | python 3.9 | 3.10 | 3.11 | 3.12 | platform macOS

mt5_mac

MetaTrader 5 native Python library for macOS.

Control MetaTrader 5 installed on your Mac directly from Python — no Windows, no remote servers, no Docker. Uses the Wine runtime bundled inside MetaTrader 5.app to run the official `MetaTrader5` Python package.

Features

- Full API matching the official MetaTrader5 package
- Account info, tick data, historical rates, symbols
- Market orders, position management, pending orders
- Deal history and order history
- Auto-setup — downloads Python + dependencies on first use
- **Indicator reading** — Heiken Ashi (open/high/low/close) computed from OHLC data
- No Windows, no Docker, no remote servers needed

Installation

```
pip install mt5_mac
```

Requirements

- **macOS** (Intel or Apple Silicon)
- **MetaTrader 5** for macOS installed from metatrader5.com
- **Python 3.9+**

On first use, the library automatically downloads Python 3.9 for Windows and installs the `MetaTrader5` pip package inside MT5's bundled Wine environment. An internet connection is required for this one-time setup (~8 MB download).

Quick Start

```python
import mt5_mac as mt5

# Initialize
if not mt5.initialize():
    print("initialize() failed - is MetaTrader 5 installed?")
    quit()

# Login
if not mt5.login(12345678, "your_password", "YourBroker-Server"):
    print("login() failed - check your credentials")
    mt5.shutdown()
    quit()

# Account info
info = mt5.account_info()
print(f"Balance: {info.balance:.2f} {info.currency}")
print(f"Equity: {info.equity:.2f}")
print(f"Leverage: 1:{info.leverage}")

# Live tick data
tick = mt5.symbol_info_tick("EURUSD")
print(f"EURUSD: bid={tick.bid} ask={tick.ask}")

# Historical rates
rates = mt5.copy_rates_from_pos("EURUSD", mt5.TIMEFRAME_M1, 0, 10)
for r in rates:
    print(f"[{r.time}] O={r.open} H={r.high} L={r.low} C={r.close}")

# Open positions
positions = mt5.positions_get()
print(f"Open positions: {len(positions) if positions else 0}")

# Place a market order
symbol = "EURUSD"
tick = mt5.symbol_info_tick(symbol)
request = {
    "action": mt5.TRADE_ACTION_DEAL,
    "symbol": symbol,
    "volume": 0.01,
    "type": mt5.ORDER_TYPE_BUY,
    "price": tick.ask,
    "sl": tick.ask - 0.0050,
    "tp": tick.ask + 0.0100,
    "deviation": 10,
    "magic": 123456,
    "comment": "mt5_mac",
    "type_time": mt5.ORDER_TIME_GTC,
    "type_filling": mt5.ORDER_FILLING_IOC,
}
result = mt5.order_send(request)
if result.retcode == mt5.RES_E_SUCCESS:
    print(f"Order placed: #{result.order} @ {result.price}")
else:
    print(f"Order failed: retcode={result.retcode}")

# Cleanup
mt5.shutdown()
```

Complete API Reference

Initialization & Connection

Function	Description
`initialize(path=None, login=None, password=None, server="", quiet=False)`	Initialize connection to MT5 terminal. Optionally log in directly.
`shutdown()`	Disconnect from MT5 and release resources.
`login(login_id, password, server="")`	Log in to a trading account.
`last_error()`	Get the last error code (always 0 in current version).

Account & Market Data

Function	Returns	Description
`account_info()`	`MqlAccountInfo`	Balance, equity, margin, leverage, server, etc.
`symbols_get(symbol=None)`	`tuple[MqlSymbolInfo]`	All available symbols, or filter by name.
`symbol_info(symbol)`	`MqlSymbolInfo`	Detailed symbol specifications (spread, digits, etc.).
`symbol_info_tick(symbol)`	`MqlTick`	Latest tick: bid, ask, last, volume, time.
`symbol_select(symbol, enable=True)`	`bool`	Add/remove symbol from MarketWatch.

Historical Data

Function	Returns	Description
`copy_rates_from_pos(symbol, tf, start, count)`	`tuple[MqlRates]`	Rates starting from a position index.
`copy_rates_from(symbol, tf, date_from, count)`	`tuple[MqlRates]`	Rates starting from a date (Unix timestamp).
`copy_rates_range(symbol, tf, date_from, date_to)`	`tuple[MqlRates]`	Rates for a date range.

Trading

Function	Returns	Description
`order_send(request)`	`MqlTradeResult`	Place/modify/close orders.
`positions_get(symbol="")`	`tuple[MqlPosition]`	Get open positions, optionally filtered by symbol.
`orders_get(symbol="")`	`tuple[MqlOrder]`	Get pending orders, optionally filtered by symbol.
`history_deals_get(from=0, to=0)`	`tuple[MqlDeal]`	Get deal history.
`history_orders_get(from=0, to=0)`	`tuple[MqlOrder]`	Get order history.

Indicators

Function	Returns	Description
`indicator_init(symbol, tf, path, buffer_count=4, params=None)`	`int`	Create indicator handle. Returns `INVALID_HANDLE` (-1) on failure.
`indicator_read(handle, buffer_num, count)`	`tuple[MqlIndicatorBuffer]`	Read latest `count` values from buffer `buffer_num`.
`indicator_deinit(handle)`	`bool`	Release indicator handle.
`indicator_list()`	`dict`	All active indicators with metadata.

Heiken Ashi constants:

- `INDICATOR_HEIKEN_ASHI` → `"Indicators\\Examples\\Heiken_Ashi.ex5"`
- `HEIKEN_ASHI_BUFFER_OPEN` (0), `HEIKEN_ASHI_BUFFER_HIGH` (1), `HEIKEN_ASHI_BUFFER_LOW` (2), `HEIKEN_ASHI_BUFFER_CLOSE` (3)

Example:

```python
ha = mt5.indicator_init("XAUUSDm", mt5.TIMEFRAME_M1, mt5.INDICATOR_HEIKEN_ASHI)
opens = mt5.indicator_read(ha, mt5.HEIKEN_ASHI_BUFFER_OPEN, 5)
for o in opens:
    print(f"time={o.time} ha_open={o.value:.5f}")
mt5.indicator_deinit(ha)
```

Constants

Timeframes: `TIMEFRAME_M1`, `M5`, `M15`, `M30`, `H1`, `H4`, `D1`, `W1`, `MN1`

Order Types: `ORDER_TYPE_BUY`, `ORDER_TYPE_SELL`, `ORDER_TYPE_BUY_LIMIT`, `ORDER_TYPE_SELL_LIMIT`, `ORDER_TYPE_BUY_STOP`, `ORDER_TYPE_SELL_STOP`

Trade Actions: `TRADE_ACTION_DEAL`, `TRADE_ACTION_PENDING`, `TRADE_ACTION_SLTP`, `TRADE_ACTION_MODIFY`, `TRADE_ACTION_REMOVE`

Order Fillings: `ORDER_FILLING_FOK`, `ORDER_FILLING_IOC`, `ORDER_FILLING_RETURN`

Order Times: `ORDER_TIME_GTC`, `ORDER_TIME_DAY`, `ORDER_TIME_SPECIFIED`

Data Classes

`MqlTick`: `time`, `bid`, `ask`, `last`, `volume`, `time_msc`, `flags`, `volume_real`

`MqlRates`: `time`, `open`, `high`, `low`, `close`, `tick_volume`, `spread`, `real_volume`

`MqlAccountInfo`: `login`, `trade_mode`, `leverage`, `balance`, `credit`, `profit`, `equity`, `margin`, `margin_free`, `margin_level`, `name`, `server`, `currency`, `company`

`MqlTradeResult`: `retcode`, `deal`, `order`, `volume`, `price`, `bid`, `ask`, `comment`, `request_id`

`MqlPosition`: `ticket`, `time`, `type`, `magic`, `identifier`, `symbol`, `volume`, `price_open`, `sl`, `tp`, `price_current`, `swap`, `profit`

`MqlOrder`: `ticket`, `time_setup`, `type`, `state`, `magic`, `symbol`, `volume_current`, `volume_initial`, `price_open`, `sl`, `tp`, `price_current`, `comment`

`MqlDeal`: `ticket`, `order`, `time`, `type`, `entry`, `magic`, `position`, `symbol`, `volume`, `price`, `commission`, `swap`, `profit`, `fee`

`MqlSymbolInfo`: `name`, `digits`, `spread`, `point`, `trade_mode`, `bid`, `ask`, `volume_min`, `volume_max`, `volume_step`, `contract_size`, `currency_base`, `currency_profit`, `description`, `path`

`MqlIndicatorBuffer`: `time`, `value`

`INVALID_HANDLE`: `-1` — returned by `indicator_init()` on failure.

How It Works

MetaTrader 5 for macOS is the Windows version of MT5 running inside a bundled Wine (CrossOver) wrapper.

```
┌──────────────────────────────────────────────────┐
│  Your Python Script (macOS)                       │
│  import mt5_mac as mt5                            │
│  mt5.initialize()                                 │
│  mt5.login(123, "pwd", "server")                  │
│  info = mt5.account_info()                        │
│  mt5.shutdown()                                   │
└──────────────────────────────────────────────────┘
                   │ JSON over stdin/stdout
                   ▼
┌──────────────────────────────────────────────────┐
│  Wine Subprocess (inside MT5.app's bundled Wine) │
│  Python 3.9 for Windows                           │
│  import MetaTrader5 as mt5                        │
│  mt5.initialize(path=...)                         │
│  mt5.login(...)                                   │
│  mt5.account_info()                               │
└──────────────────────────────────────────────────┘
                   │ MT5 API
                   ▼
┌──────────────────────────────────────────────────┐
│  MetaTrader 5 Terminal (Windows binary in Wine)  │
│  terminal64.exe                                   │
│  Connected to broker                              │
└──────────────────────────────────────────────────┘
```

First-Run Setup

On the very first call to `initialize()`, the library:

1. Locates `MetaTrader 5.app` in `/Applications`
2. Finds the bundled Wine binaries
3. Downloads **Python 3.9 embeddable for Windows** (~8 MB) from python.org
4. Extracts it to the appropriate Wine prefix
5. Installs **pip** and the **MetaTrader5** package
6. Launches the agent and connects to MT5

This setup happens once. Subsequent connections are instant.

Project Structure

```
mt5_mac/
├── mt5_mac/
│   ├── __init__.py       # Top-level API (initialize, login, order_send, etc.)
│   ├── agent.py          # Agent script that runs inside Wine
│   ├── backend.py        # Wine subprocess communication + auto-bootstrap
│   ├── bootstrap.py      # Auto-download Python & MetaTrader5 in Wine
│   ├── indicators.py     # Pure-Python indicator computations (Heiken Ashi)
│   ├── types.py          # Data classes (MqlTick, MqlRates, etc.)
│   └── bridge/           # MQL5 source files (for future use)
├── examples/
│   ├── basic.py           # Basic usage example
│   ├── streaming.py       # Real-time tick streaming
│   └── trading.py         # Market order placement
├── pyproject.toml         # Package configuration
├── setup.py               # Legacy setup for older pip
├── LICENSE                # MIT License
└── README.md              # This file
```

Example Scripts

Basic: `examples/basic.py`

```python
import mt5_mac as mt5

mt5.initialize()
mt5.login(12345678, "password", "Broker-Server")

info = mt5.account_info()
print(f"{info.login} @ {info.server} | ${info.balance:.2f}")

tick = mt5.symbol_info_tick("EURUSD")
print(f"EURUSD: bid={tick.bid} ask={tick.ask}")

mt5.shutdown()
```

Streaming: `examples/streaming.py`

```python
import time
import mt5_mac as mt5

mt5.initialize()
mt5.login(12345678, "password", "Broker-Server")

symbols = ["EURUSD", "GBPUSD", "USDJPY"]
for sym in symbols:
    mt5.symbol_select(sym, True)

try:
    while True:
        for sym in symbols:
            tick = mt5.symbol_info_tick(sym)
            if tick:
                print(f"{sym}: {tick.bid:.5f} / {tick.ask:.5f}")
        time.sleep(1)
except KeyboardInterrupt:
    pass
finally:
    mt5.shutdown()
```

Trading: `examples/trading.py`

```python
import mt5_mac as mt5

mt5.initialize()
mt5.login(12345678, "password", "Broker-Server")

tick = mt5.symbol_info_tick("EURUSD")
if not tick:
    quit()

request = {
    "action": mt5.TRADE_ACTION_DEAL,
    "symbol": "EURUSD",
    "volume": 0.01,
    "type": mt5.ORDER_TYPE_BUY,
    "price": tick.ask,
    "sl": tick.ask - 0.0050,
    "tp": tick.ask + 0.0100,
    "deviation": 10,
    "magic": 123456,
    "comment": "mt5_mac",
    "type_time": mt5.ORDER_TIME_GTC,
    "type_filling": mt5.ORDER_FILLING_IOC,
}
result = mt5.order_send(request)
print(f"Order: #{result.order} retcode={result.retcode}")

mt5.shutdown()
```

Troubleshooting

"MetaTrader 5.app not found"

Download and install MetaTrader 5 from metatrader5.com. After installation, the app should be at `/Applications/MetaTrader 5.app`.

"Failed to download Python"

The first run downloads Python 3.9 embeddable for Windows (~8 MB). Check your internet connection. The download is cached after extraction.

"Login failed"

Verify your account credentials (login ID, password, server name). Some brokers require the full server name (e.g. `Exness-MT5Trial15`, not just `Exness`).

"Empty response from backend"

This usually means the MT5 terminal couldn't start inside Wine. Try launching `MetaTrader 5.app` manually once, let it fully load, then try again.

License

MIT License — see LICENSE for details.

Author

Abdul Rafay Jiwani

Built for macOS · No Windows required · No Docker · No remote servers

Project links

- Homepage
- Repository

Key dates

PyPI data ›

Released: Jul 15, 2026

LATEST RELEASE

1 maintainer

PyPI data ›

rafayjiwani

Credits

Author: Abdul Rafay Jiwani

License

MIT License (MIT)

Requires

Python >=3.9

Tags

`mt5` `metatrader5` `trading` `forex` `macos` `metatrader`

Classifiers

Development Status
- 4 - Beta

Intended Audience
- Developers
- Financial and Insurance Industry

License
- OSI Approved :: MIT License

Operating System
- MacOS :: MacOS X

Programming Language
- Python :: 3
- Python :: 3.9
- Python :: 3.10
- Python :: 3.11
- Python :: 3.12

Topic
- Office/Business :: Financial :: Investment
- Software Development :: Libraries :: Python Modules

Cubist Systematic Strategies

Cubist Systematic Strategies is a Contributing sponsor of the Python Software Foundation.

PSF Sponsor · Served ethically

PyPI

Developed and maintained by the Python Software Foundation and Python community, for the Python community.

Status: All Systems Operational

Donate today!

Help
- PyPI documentation
- Getting help
- Installing packages
- Uploading packages
- Packaging user guide

About PyPI
- PyPI blog
- Infrastructure dashboard
- Statistics
- Logos & trademarks
- Our sponsors

Contributing to PyPI
- Bugs and feedback
- Contribute on GitHub
- Translate PyPI
- Sponsor PyPI
- Development credits

Using PyPI
- Terms of Service
- Report security issue
- Code of conduct
- Privacy Notice
- Acceptable Use Policy

Anthropic, PBC	Visionary sponsor
Bloomberg	Visionary sponsor
Hudson River Trading	Visionary sponsor
Meta	Visionary sponsor
NVIDIA	Visionary sponsor
Microsoft	Sustainability sponsor
Depot	Continuous Integration
AWS	Cloud computing and Security Sponsor
Datadog	Monitoring
Fastly	CDN
Google	Download Analytics
Sentry	Error logging
StatusPage	Status page

"PyPI", "Python Package Index", and the blocks logos are registered trademarks of the Python Software Foundation. © 2026 Python Software Foundation Site map Deployed from `ce99a6d`